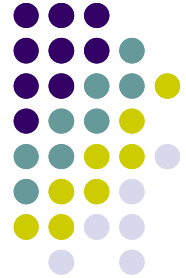


Reference Design Patterns

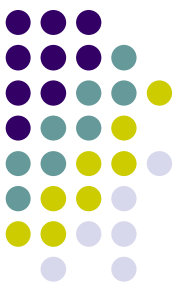


Architecture of Computer Games
SE 456

Ed Keenan

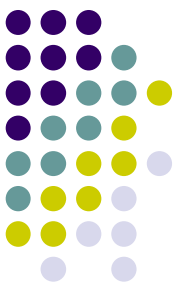
10 March 2021

13.0.8.6.1 Mayan Long Count



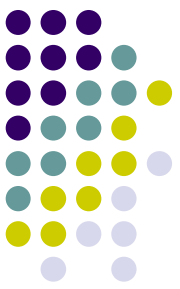
Reference Material

- References
- Websites
 - https://sourcemaking.com/design_patterns
 - <http://www.oodesign.com/>
 - <http://www.dofactory.com/net/design-patterns>
- Books
 - Head First Design Patterns
 - Design Patterns – explained simply



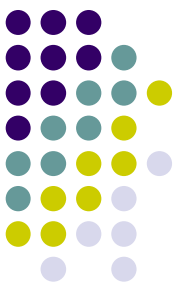
Categories

- Creational design patterns
 - These design patterns are all about class instantiation.
 - This pattern can be further divided into class-creation patterns and object-creational patterns.
 - While class-creation patterns use inheritance effectively in the instantiation process, object-creation patterns use delegation effectively to get the job done.
- Structural design patterns
 - These design patterns are all about Class and Object composition.
 - Structural class-creation patterns use inheritance to compose interfaces.
 - Structural object-patterns define ways to compose objects to obtain new functionality.
- Behavioral design patterns
 - These design patterns are all about Class's objects communication.
 - Behavioral patterns are those patterns that are most specifically concerned with communication between objects.



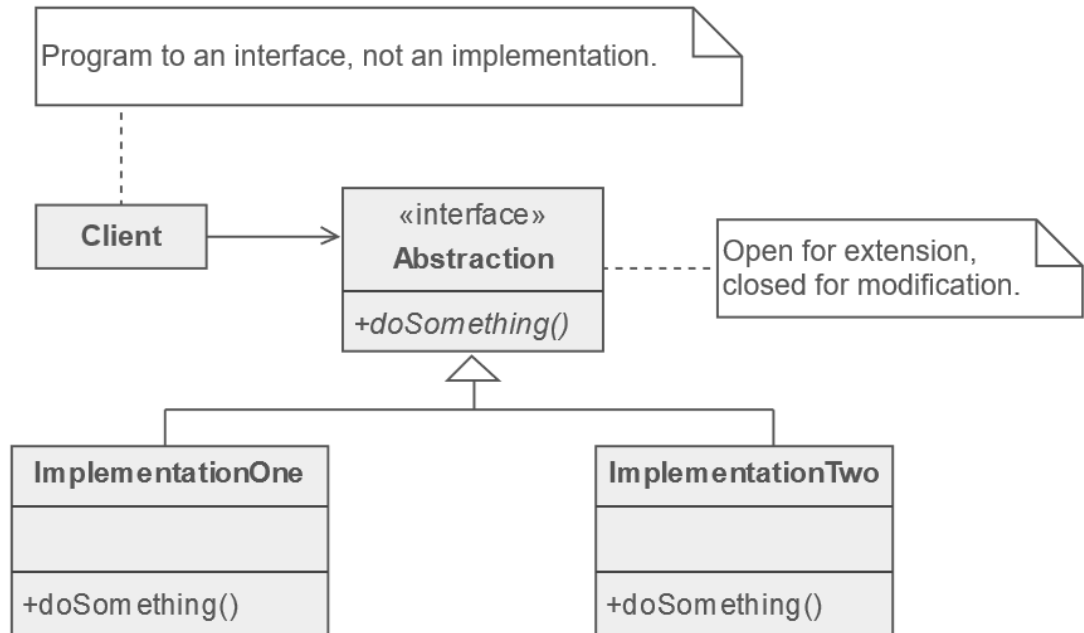
Categories

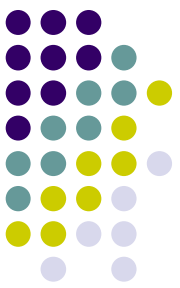
- Creational patterns
 - Object Pool
 - Factory
 - Singleton
- Structural patterns
 - Adaptor
 - Composite
 - Flyweight
 - Proxy
- Behavioral patterns
 - Null Object
 - Observer
 - Command
 - Iterator
 - State
 - Strategy
 - Visitor
 - Template



Strategy

- Intent:
 - Define a family of algorithms, encapsulate each one, and make them interchangeable.
 - Strategy lets the algorithm vary independently from the clients that use it.
- Remarks
 - Capture the abstraction in an interface, bury implementation details in derived classes.





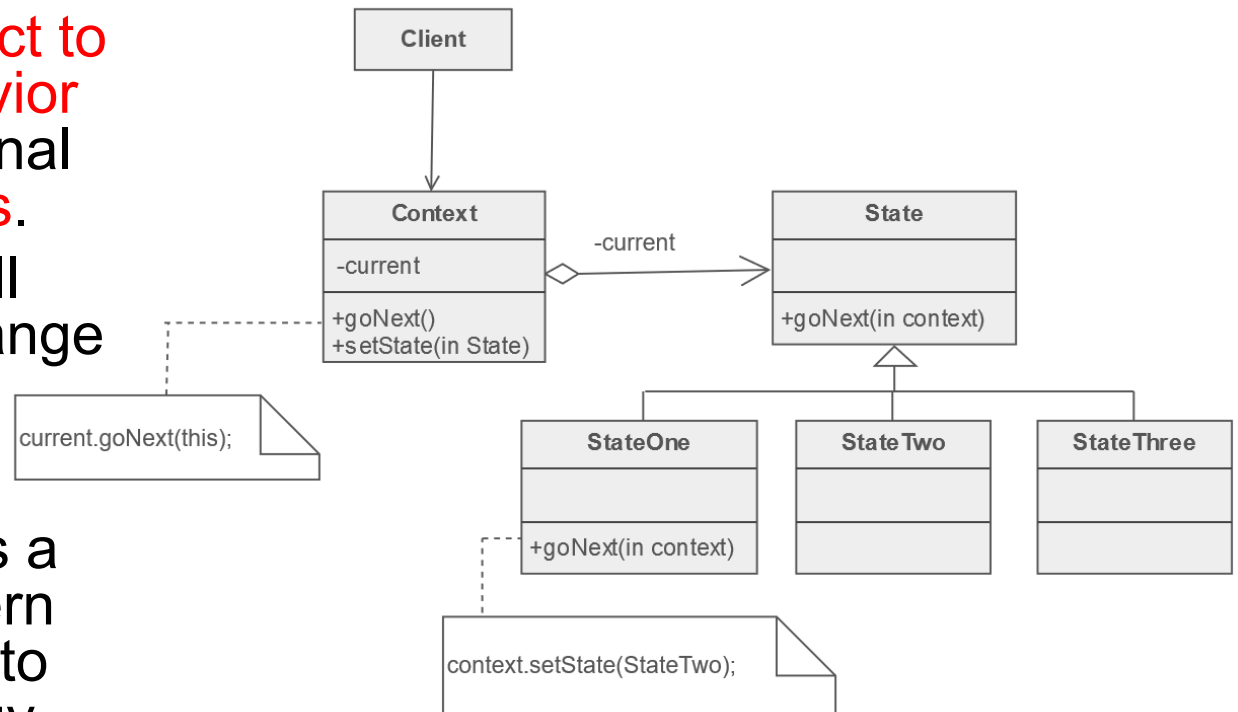
State

- Intent

- Allow an object to alter its behavior when its internal state changes.
- The object will appear to change its class.

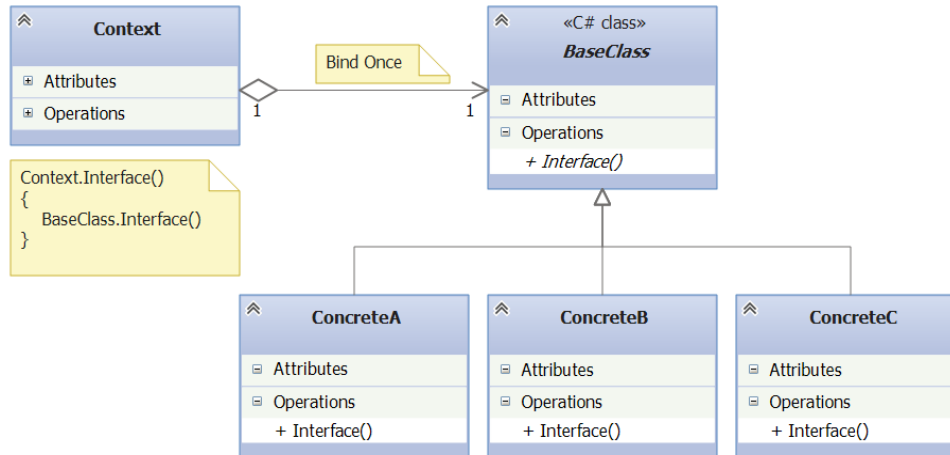
- Remark

- Interpreted as a strategy pattern which is able to switch strategy through its methods.

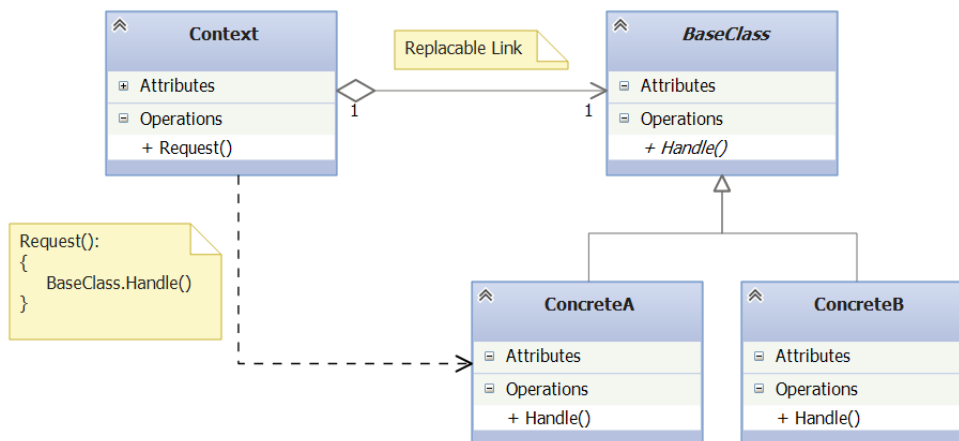


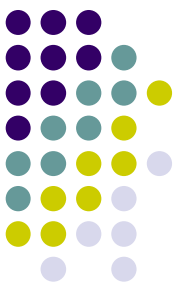


Strategy vs State

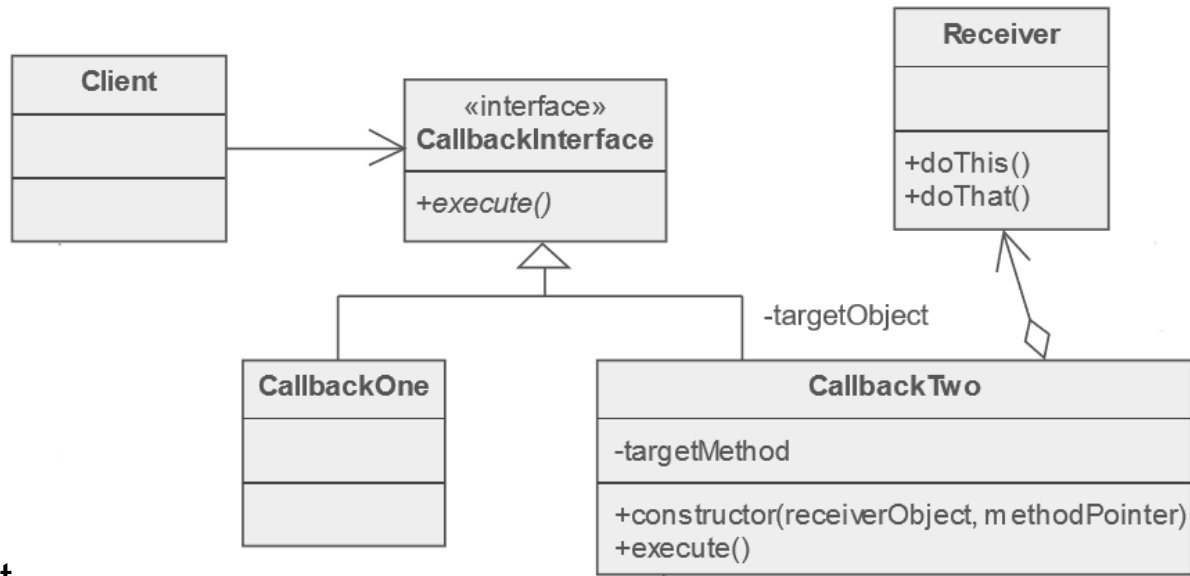


- *Differ in their intent*
- **Strategy Pattern** as a flexible alternative to subclassing
 - if you use inheritance to define the behavior of a class,
 - stuck with that behavior
 - Strategy you can change the behavior by composing with a different object
- **State Pattern** as an alternative to putting lots of conditionals in your context
 - encapsulating the behaviors within state objects
 - change the state object in context to change its behavior





Command

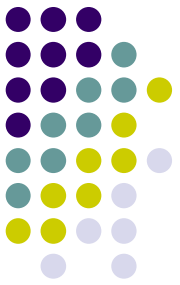


- Intent

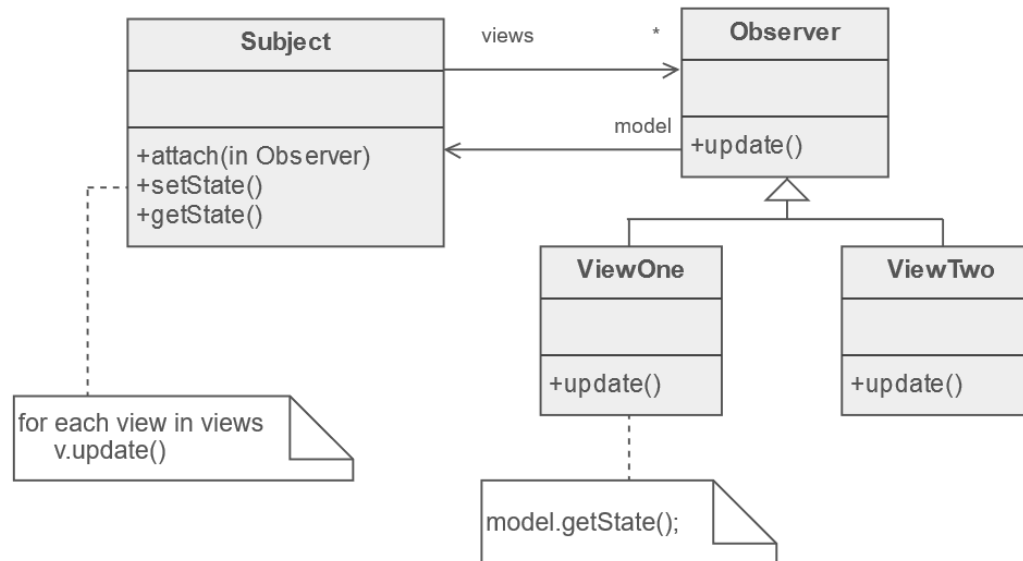
- Encapsulate a request as an object, thereby letting you parametrize clients with different requests, queue or log requests, and support undoable operations.
- Promote "invocation of a method on an object" to full object status

- Remark

- An object-oriented callback
- Command is used to model an action you need more to do with than just execute it now.
 - You can hand commands around, remember them in a list to be able to do them again in the correct order, you can add an undo action etc.



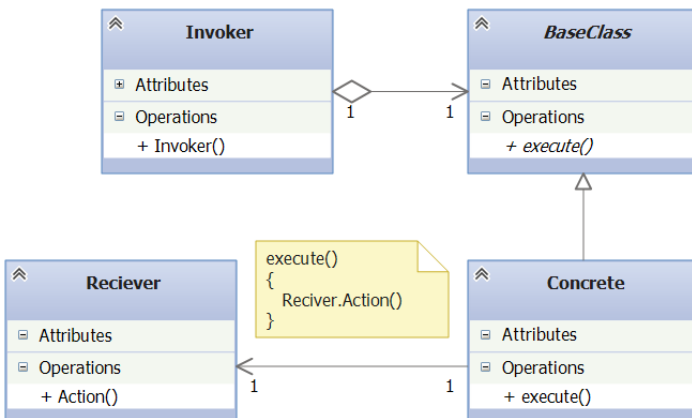
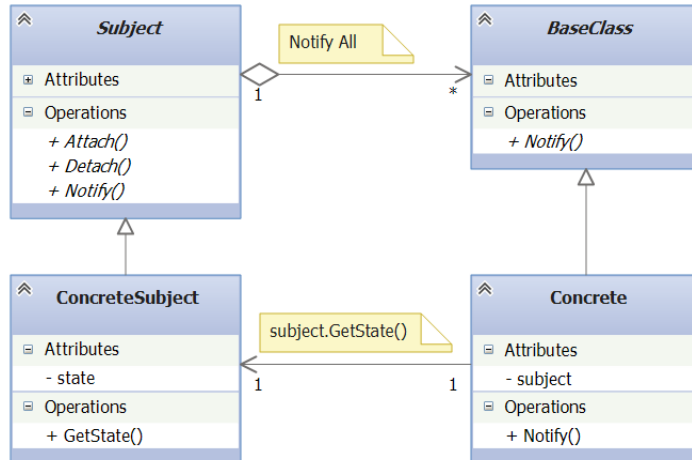
Observer



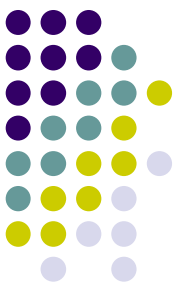
- **Intent**
 - Define a **one-to-many dependency** between objects so that when one object changes state, **all its dependents are notified** and updated automatically.
- **Remark**
 - **Observer** is used when the Observable needs to notify other objects about certain events, but doesn't know **which** objects to notify.
 - Similar structure as Command



Observer vs Command

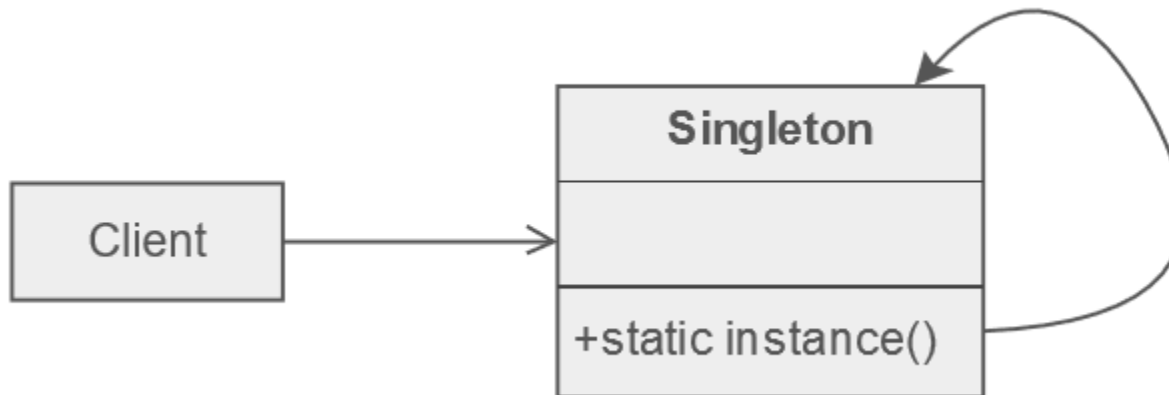


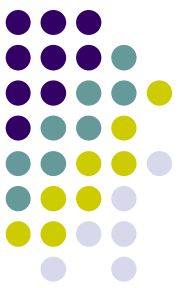
- *Differ in their intent*
- **Observer** pattern is used when the **Listeners** needs to notify other objects about certain events, **but doesn't know** *which* objects to notify.
 - Reusable – notify again and again
 - Stay stationary on the list
- **Command** pattern is used to model an **action** you need more to do with than just execute it now.
 - You can hand commands around, remember them in a list to be able to do them again in the correct order, you can add an undo action



Singleton

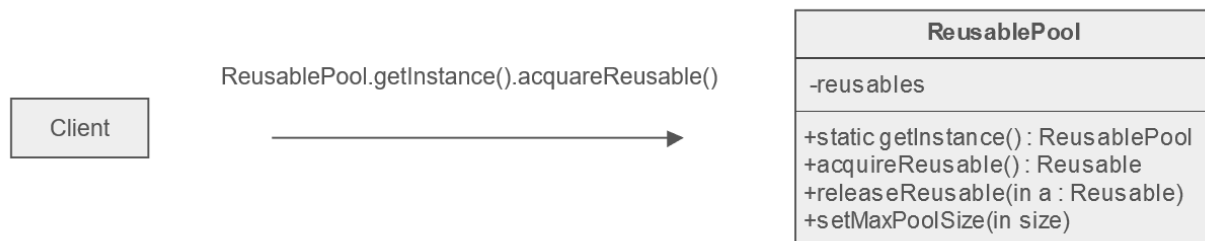
- Intent
 - Ensure a class has only **one instance**, and provide a global point of access to it.
- Remark
 - Encapsulated "just-in-time initialization" or "initialization on first use".

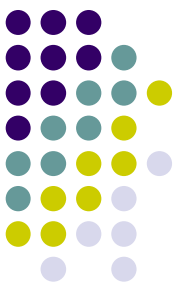




Object Pooling

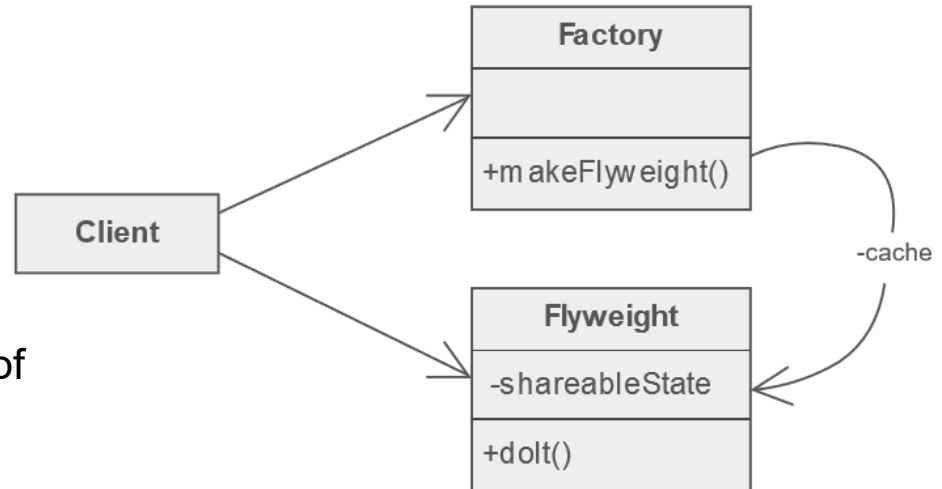
- Intent
 - Reuse and share objects that are expensive to create.
- Remark
 - Best use: cost of initializing a class instance is high, the rate of instantiation of a class is high, and the number of instantiations is finite

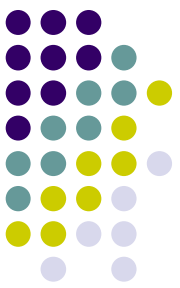




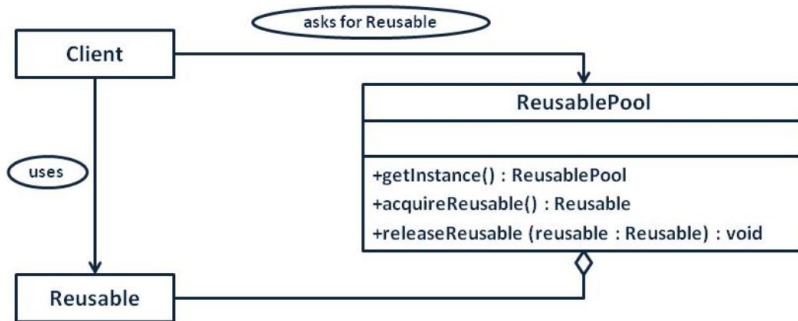
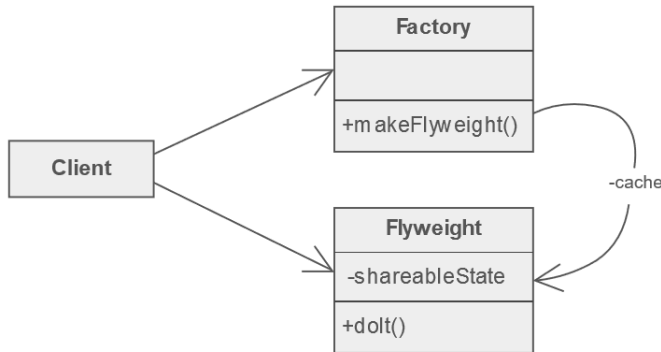
Flyweight

- Intent
 - Use **sharing** to support large numbers of **fine-grained objects** efficiently.
- Remark
 - Use sharing to support a large number of objects that have part of their internal state in common where the other part of state can vary.
 - Each "flyweight" object is divided into two pieces: state-dependent (extrinsic) part, and the state-independent (intrinsic) part.
 - Intrinsic state is stored (shared) in the Flyweight object.
 - Extrinsic state is stored or computed by client objects, and passed to the Flyweight when its operations are invoked.





Flyweight vs Pool

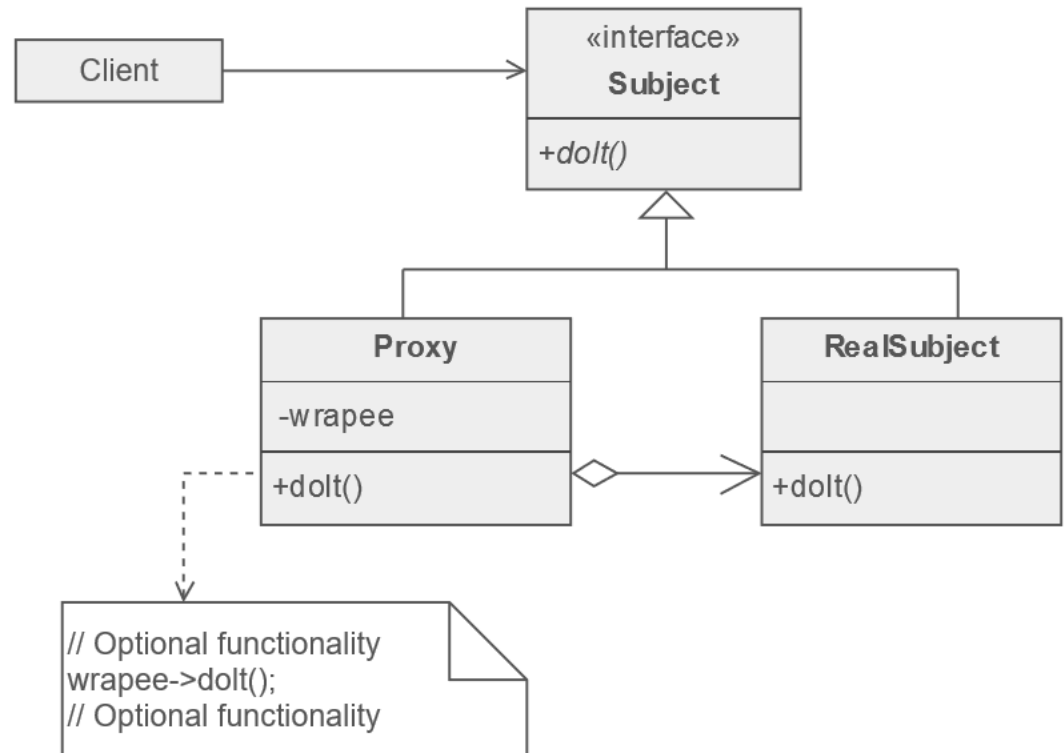


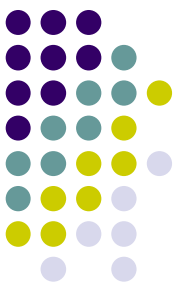
- *Differ How objects are used*
- **Flyweight** resources are immutable
 - Flyweight many clients can simultaneously use the same object.
- **Object pool** are mutable.
 - Object pool, at any given moment the object can be accessed only for one client
- Categorize the following:
 - Texture, Image, Sprite



Proxy

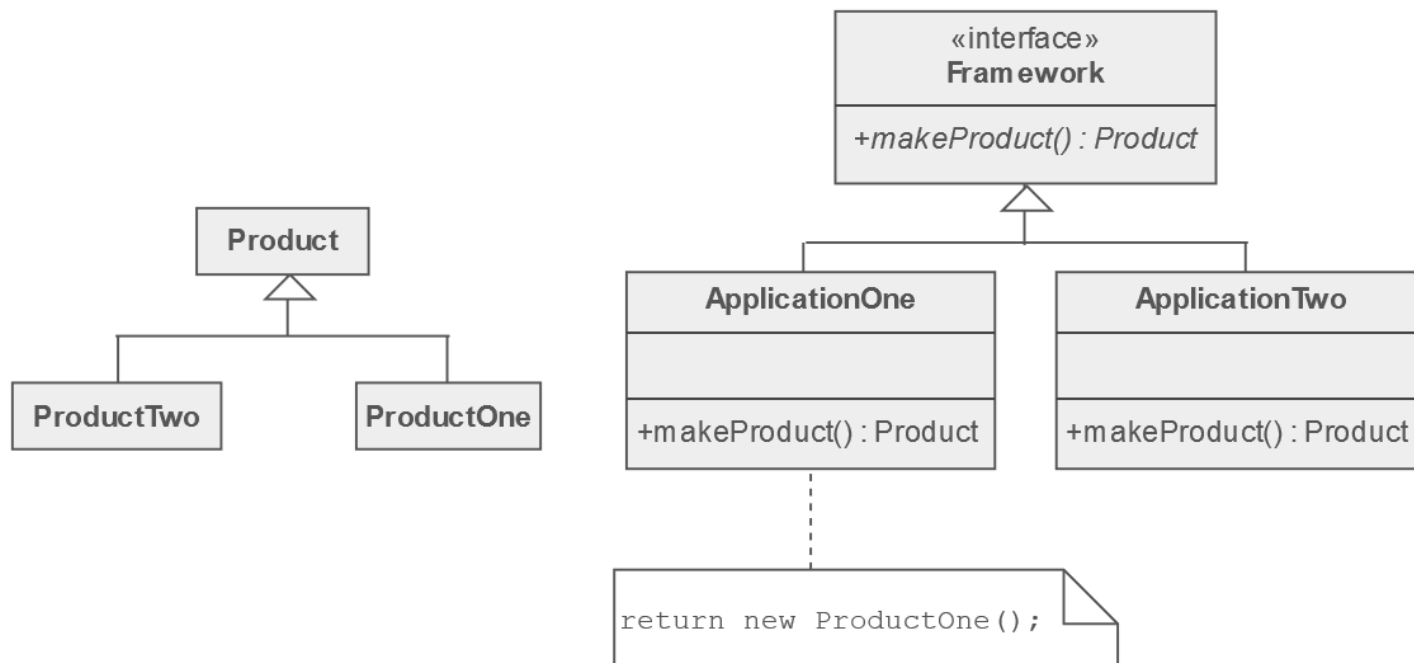
- Intent
 - Provide a **surrogate** or placeholder for another object to control access to it.
- Remark
 - Use an extra level of indirection to support distributed, controlled, or intelligent access.
 - Add a wrapper and delegation to protect the real component from undue complexity.
 - **Instance of smaller object** using a full feature object

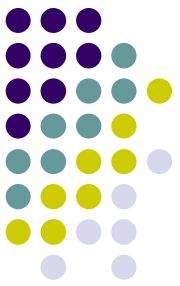




Factory

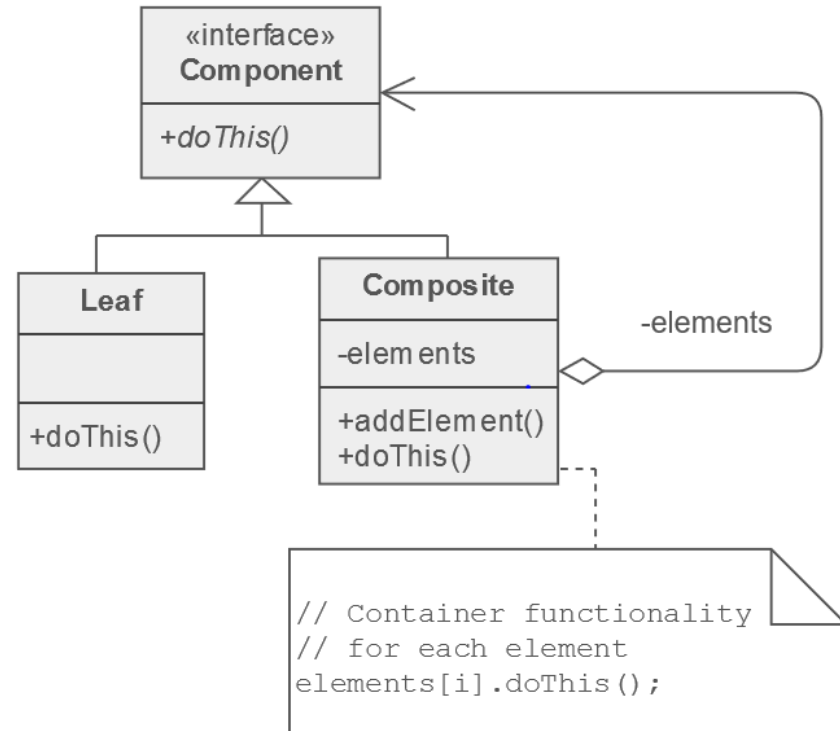
- Intent
 - Creates objects without exposing the instantiation logic to the client.
 - Refers to the newly created object through a common interface

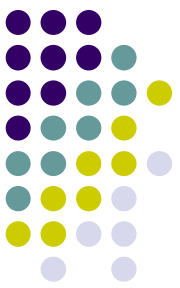




Composite

- Intent
 - Compose objects into tree structures to represent whole-part hierarchies.
 - Composite lets clients treat individual objects and compositions of objects uniformly.
- Remarks
 - Recursive composition
 - 1-to-many "has a" up the "is a" hierarchy

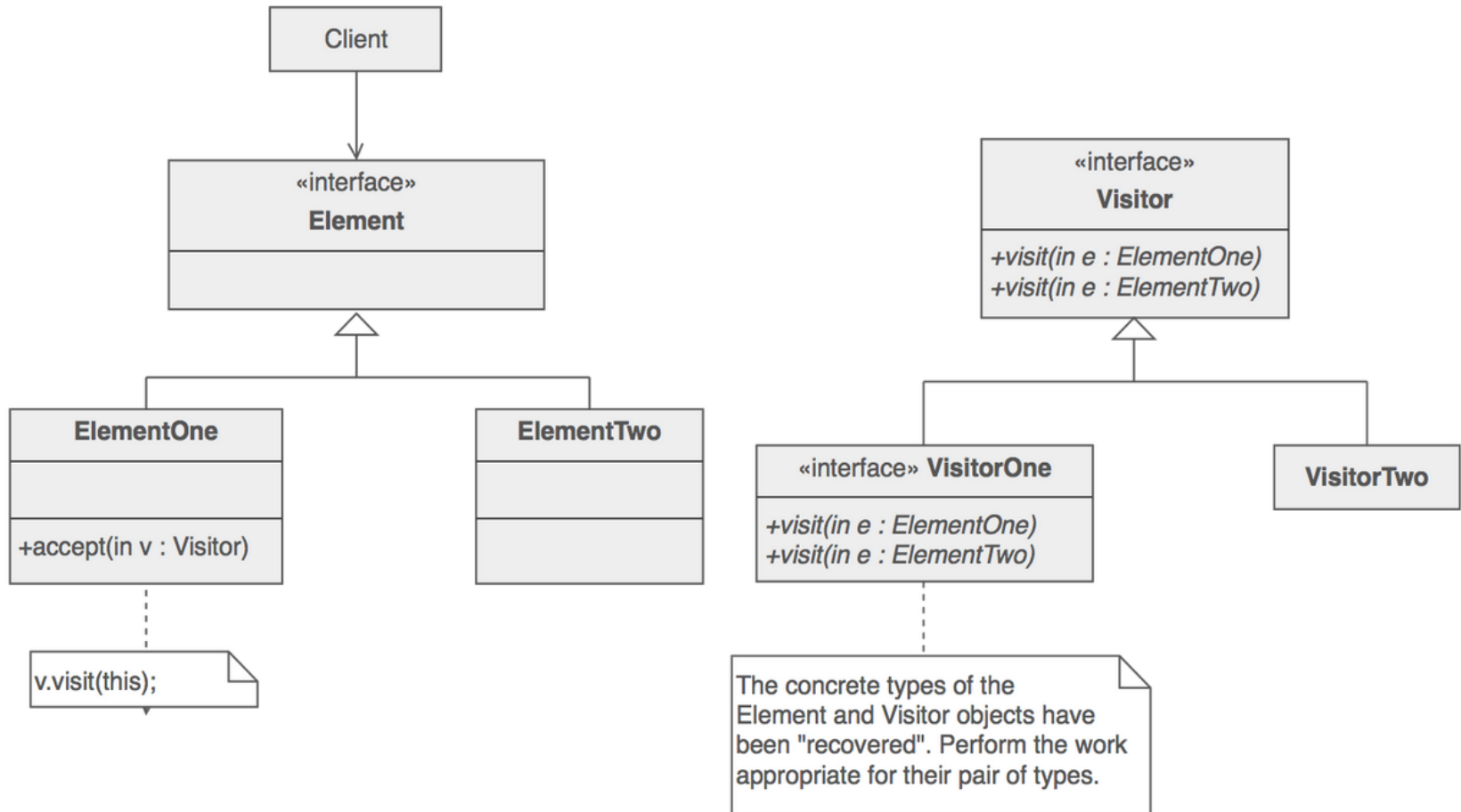
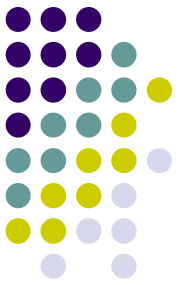




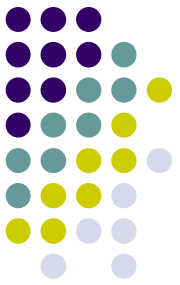
Visitor

- Intent
 - Represent an operation to be performed on the elements of an object structure.
 - Visitor lets you define a new operation without changing the classes of the elements on which it operates.
- Remarks
 - The classic technique for recovering lost type information.
 - Do the right thing based on the type of two objects.
 - Double dispatch

Visitor



Adaptor

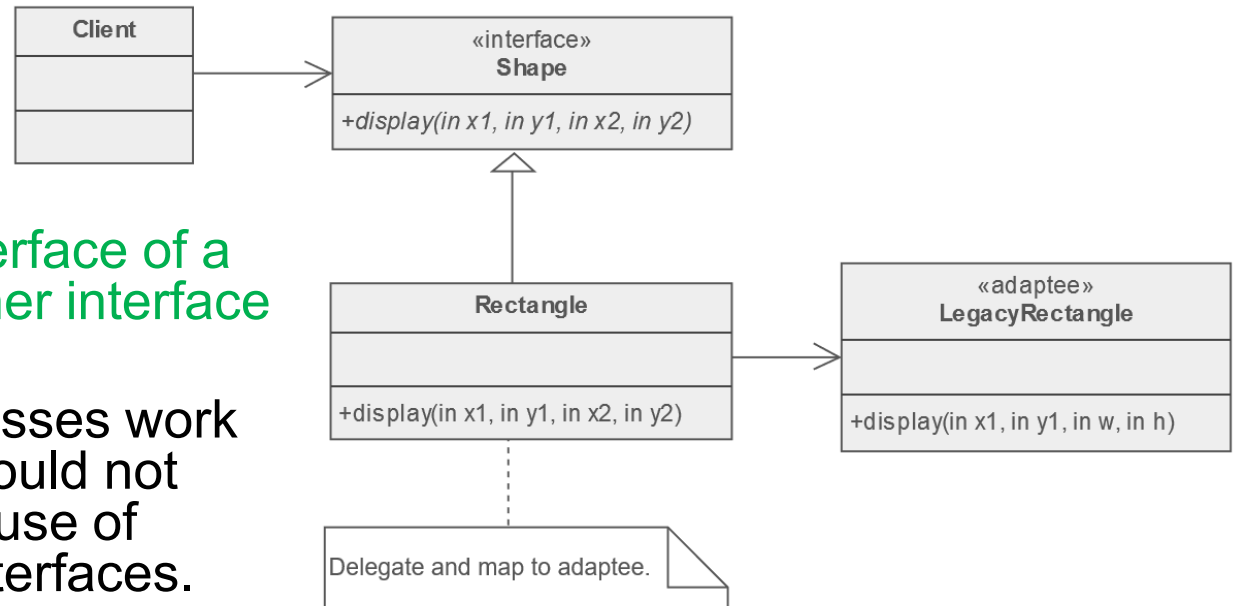


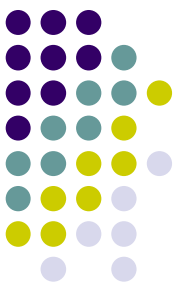
- Intent

- Convert the interface of a class into another interface clients expect.
- Adapter lets classes work together, that could not otherwise because of incompatible interfaces.

- Remarks

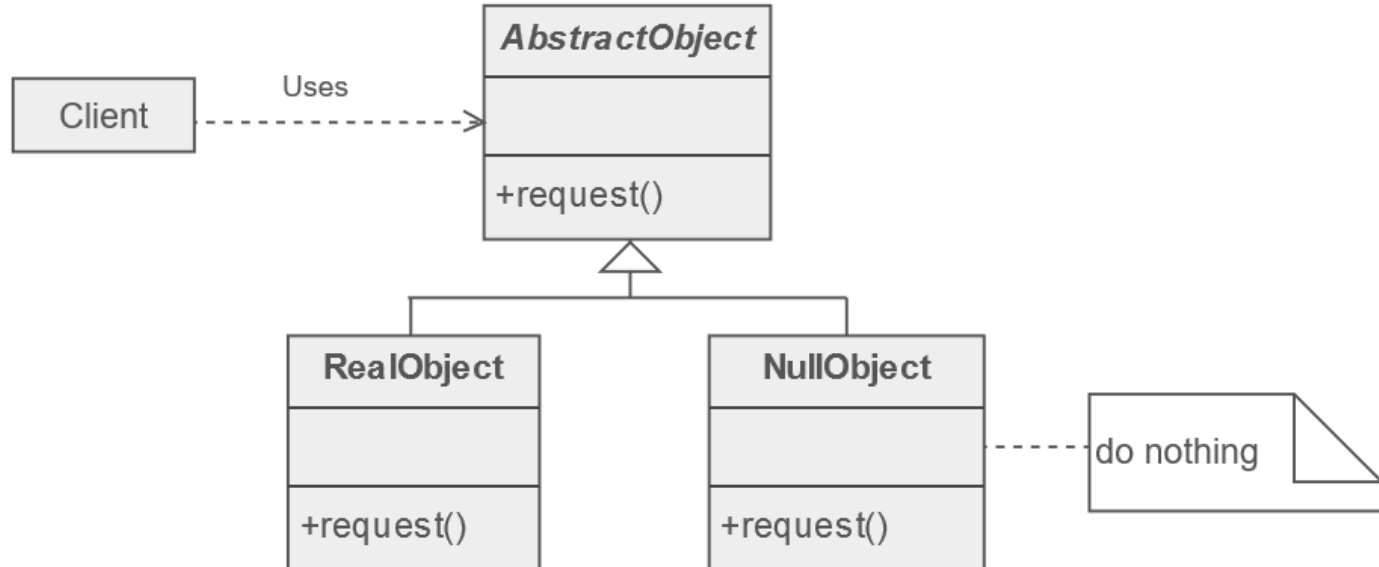
- **Wrap** an existing class with a new interface.
- Impedance match an old component to a new system

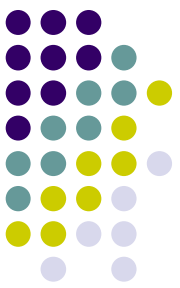




Null Object

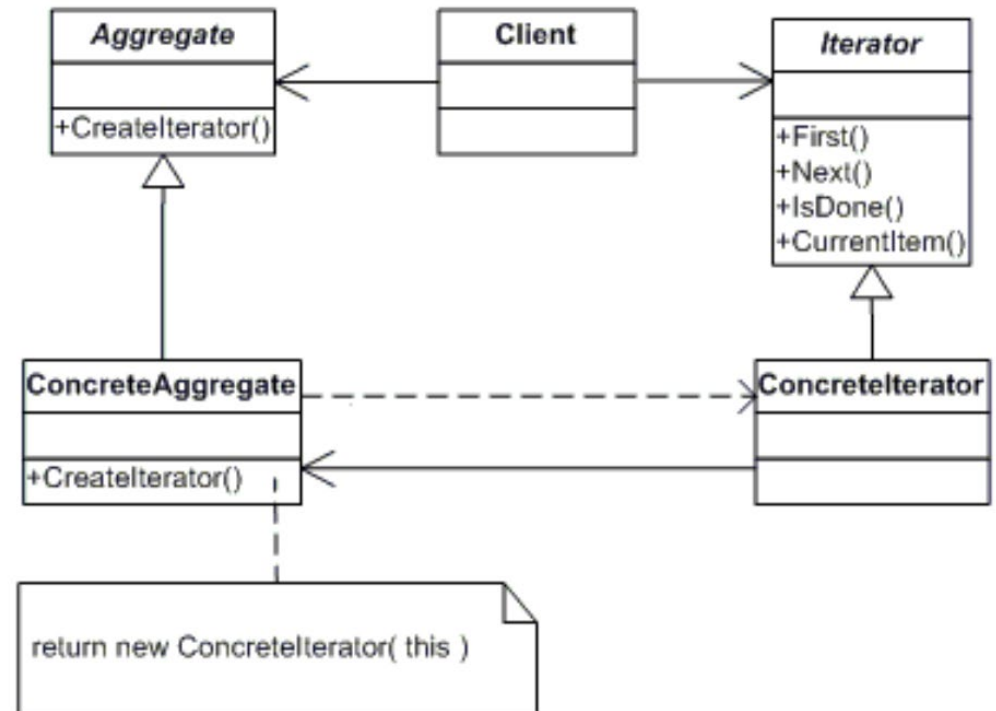
- Intent
 - Provide an object as a **surrogate for the lack of an object** of a given type.
 - The Null Object Pattern provides intelligent **do nothing behavior**, hiding the details from its collaborators.
- Remarks
 - The intent of a Null Object is to encapsulate the absence of an object by providing a substitutable alternative that offers suitable default do nothing behavior.

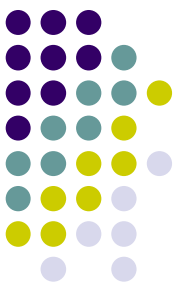




Iterator

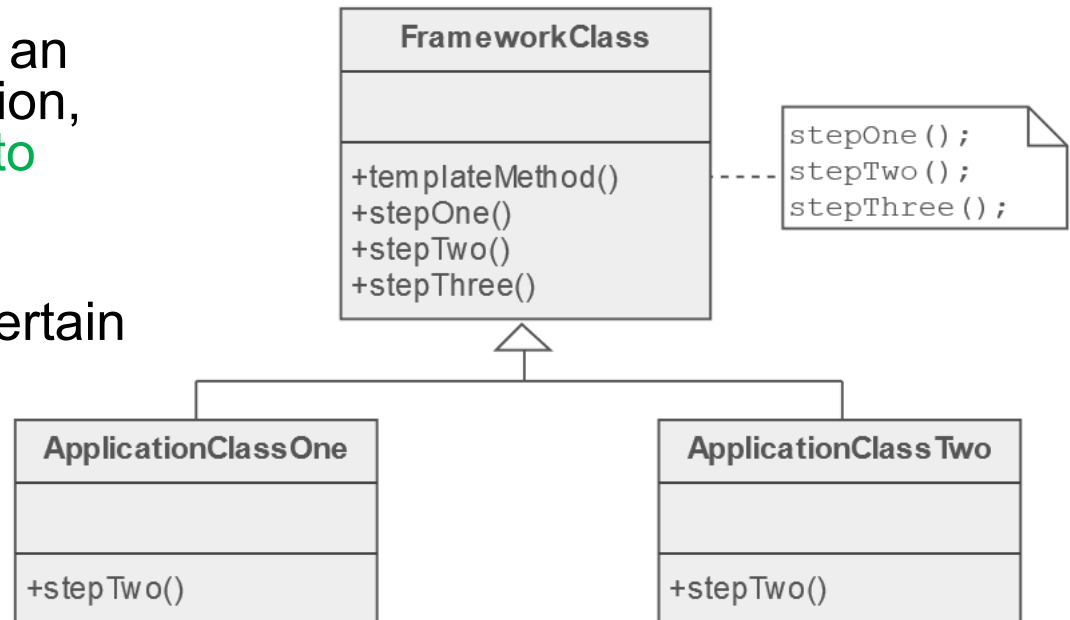
- Intent
 - Provide a way to **access** the elements of an aggregate object **sequentially without exposing its underlying representation**.
 - Provides an abstraction that makes it possible to decouple collection classes and algorithms.
- Remarks
 - Promote to "full object status" the traversal of a collection.
 - Polymorphic traversal



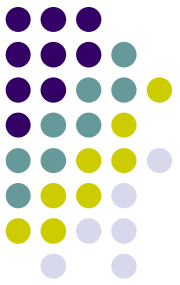


Template

- Intent
 - Define the skeleton of an **algorithm** in an operation, **deferring some steps to client subclasses**.
 - Template Method lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.
- Remarks
 - Base class declares algorithm 'placeholders', and derived classes implement the placeholders.



Thank You!



- Questions?

